MEDIUM

Three Misbehavior Detection Code Paths Silently Drop Byzantine Behaviour

Repo: ton-blockchain/ton **Branch:** testnet **Commit:** `3bb6abc` **Scope:** Primary — validator/consensus/simplex/consensus.cpp **Date:** 2026-03-14

Summary

Three separate code paths in `consensus.cpp` detect a Byzantine validator behaviour and then do nothing — the violations are logged (in some cases not even that) and execution continues. Each location contains a `// FIXME: Report misbehavior` comment acknowledging the omission. Byzantine leaders can exploit all three silently dropped cases to degrade liveness without facing any on-chain consequence.

The Three Locations

1. Candidate with invalid parent structure (consensus.cpp:174)

```
// consensus.cpp — CandidateReceived handler, line 172–175
if (candidate->parent_id.has_value()
    && candidate->parent_id->slot >= candidate->id.slot) {
  // FIXME: report misbehavior
  return;
}
```

A candidate proposing a parent whose slot index is not strictly less than its own slot is structurally invalid — the parent would be at the same height or ahead of the candidate in the chain. The validator correctly rejects it, but files no misbehavior report. The leader that sent this candidate faces no penalty.

2. Leader proposes two different candidates for the same slot (consensus.cpp:180)

```
// consensus.cpp — CandidateReceived handler, line 177–181
if (slot->state->pending_block.has_value()) {
  if (slot->state->pending_block.value()->id != candidate->id) {
    // FIXME: Report misbehavior
  }
  return;
}
```

If a second, different candidate for the same slot arrives from the same leader, this is proposal equivocation — the leader is broadcasting two conflicting blocks. The validator silently ignores the duplicate. No misbehavior report is filed, and the leader can repeat this freely to waste validator resources (re-triggering validation on each candidate) with no deterrent.

3. Block candidate fails validation (consensus.cpp:227)

```
// consensus.cpp — try_notarize(), line 224–228
if (validation_result.has<CandidateReject>()) {
  LOG(WARNING) << "Candidate " << candidate->id
               << " is rejected: " << validation_result.get<CandidateReject>().reason;
  // FIXME: Report misbehavior
  co_return {};
}
```

When full block validation rejects a candidate (`CandidateReject`), the rejection is logged but not reported as misbehavior. A Byzantine leader can submit invalid blocks — blocks that pass superficial checks but fail validation — to occupy validators' validation slots (the `ValidationRequest` co_await), slow consensus, and consume bandwidth without facing any slashing risk.

Combined Impact

Location	Line	Byzantine Action	Effect on Network
Parent slot check	174	Propose candidate with parent_slot ≥ own slot	Wastes processing; no penalty
Duplicate candidate	180	Send two different blocks for same slot	Forces re-validation; no penalty
Invalid block	227	Send block that passes early checks but fails full validation	Occupies ValidationRequest slot; slows consensus; no penalty

These three paths represent a systematic gap in the misbehavior accountability layer. Individually, each is a moderate nuisance. Combined, a Byzantine leader can cycle through all three strategies to maximise disruption while remaining invisible to the slashing system.

Fix

Implement misbehavior reporting at each location. For cases 1 and 2, the evidence is available in scope (the candidate itself and, for case 2, the earlier `pending_block`). For case 3, the `CandidateReject` reason and the candidate are both available:

```
// Case 2 fix — dual-candidate equivocation
if (slot->state->pending_block.value()->id != candidate->id) {
  owning_bus().publish<ReportMisbehavior>(
    ProposalEquivocation::create(slot->state->pending_block.value(),
candidate)).start().detach();
  return;
}

// Case 3 fix — invalid block proposal
if (validation_result.has<CandidateReject>()) {
  owning_bus().publish<ReportMisbehavior>(
    InvalidCandidateProposal::create(candidate, validation_result.get<CandidateReject>
())).start().detach();
  co_return {};
}
```

TON Blockchain Contest Round 2 — Primary scope finding. Submitted by TesseraeVentures.